

TP 1 : Certification

ISC_125 – Sécurité des systèmes d'information et cryptographie

Ludovico Grabau et Valentin Acevedo

02.03.2021

Table des matières

De quoi s'agit-il	2
Démarche, résultats et réponses.....	2
Pré-requis :	2
Première étape :	3
Deuxième étape :	4
Troisième étape :	4
Quatrième étape :	5
Cinquième étape :	6
Sixième étape :	7
Conclusion	7

De quoi s'agit-il

Dans ce TP nous allons à travers plusieurs étapes, sécuriser un serveur Web que le service d'Apache2 met à disposition en créant et mettant en place divers types de certificats et d'autres méthodes permettant de garantir leur fonctionnement. Pour exemplifier cela, on fait passer un service web qui est initialement en `http://localhost` en `https://<ServerName>`.

Démarche, résultats et réponses

Pré-requis :

Installer Ubuntu à travers une VM ou en formatant son disque dur pour avoir un dual boot avec Ubuntu (installer Ubuntu et le remplacer par l'OS courant est également possible).

Vérifier que Ubuntu et ses fonctionnalités soient à jour à l'aide de la commande :

(→ `sudo apt-get update`)

→ `sudo apt-get upgrade`

Installer Apache2 à l'aide de la commande :

→ `sudo apt-get install apache2`

Pour faciliter la complétion du TP, il est conseillé de passer en administrateur pour éviter de réécrire "sudo" et son mot de passe à chaque fois qu'une commande requiert des permissions supérieures, pour faire cela on utilise :

→ `sudo su root`

Accéder au répertoire d'Apache2 pour pouvoir travailler sur le projet, créer les divers fichiers demandés, naviguer dans le répertoire pour modifier certains documents et initialiser Apache2.

[IMPORTANT] La longueur de tous les fichiers à créer si demander est de 2048 bit car plus tard dans le projet, si cela n'est pas fait, une erreur comme quoi la longueur de la clef est trop courte peut arriver.

Première étape :

La création des certificats (fichier.*)

a. Création du certificat serveur

- Génération de la clé privée
→ `openssl genrsa -aes256 -passout pass:1234 -out fichier.key 2048`
→ `chmod 600 fichier.key`

1) Quel est le résultat en sortie de cette commande ?

```
root@LAPTOP-31HU7CRF:/etc/apache2/ssl# openssl genrsa -aes256 -passout pass:1234 -out fichier.key 2048
Generating RSA private key, 2048 bit long modulus (2 primes)
.....+++++
.....+++++
e is 65537 (0x010001)
root@LAPTOP-31HU7CRF:/etc/apache2/ssl# chmod 600 fichier.key
```

2) Quelle est l'option à ajouter pour que cette clé soit protégée par un mot de passe ? Testez-cette commande et ensuite lancez la commande pour supprimer le mot de passe systématique.

`-aes256 -passout -pass:[PASSWORD]`

b. Création d'un fichier de demande de signature de certificat (CSR Certificate Signing Request).

- Générer la demande de certificat
→ `hostname`

[IMPORTANT] Le CommonName de l'étape de la création d'un fichier de demande de signature de certificat (CSR) doit être identique au hostname du système, il est possible d'identifier le hostname dans le terminal (ubuntu) avec la commande "hostname".

→ `openssl req -new -key fichier.key > fichier.csr`

3) Quel est l'effet (fichier de création) de cette commande ? Quel est l'élément requis pour cette création de fichier ?

Elle permet de créer un fichier de demande de signature de certificat. On a besoin de la clef contenu dans « fichier.key ».

4) Pour la suite, vous allez auto-signer votre certificat. Quelle est l'autre possibilité pour la signature du certificat ?

Directement demander une signature de certificat à une entreprise de certification.

Deuxième étape :

Création du certificat de l'autorité de certification (ca.*)

Signer un certificat (devenir votre propre autorité de certification)

- Réaliser une clé et un certificat auto-signé
- Créez la clé privée de votre autorité de certification
→ `openssl genrsa -aes256 -passout pass:1234 -out ca.key 2048`

[IMPORTANT] Mettre un nom différent que celui mis pour CommonName dans l'étape précédente qui été utilisé pour fichier.csr

→ `chmod 600 ca.key`

- Une fois la clé privée obtenue, on peut s'en servir pour générer un certificat auto-signé X509 (valide 1 an)
→ `openssl req -new -x509 -days 365 -key ca.key > ca.crt`

5) Que vous demande le système ? Justifier.

```
root@LAPTOP-31HU7CRF:/etc/apache2/ssl# openssl req -new -x509 -days 365 -key ca.key > ca.crt
Enter pass phrase for ca.key:
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:CH
State or Province Name (full name) [Some-State]:Geneva
Locality Name (eg, city) []:Geneva
Organization Name (eg, company) [Internet Widgits Pty Ltd]:HEPIA
Organizational Unit Name (eg, section) []:.
Common Name (e.g. server FQDN or YOUR name) []:iceStorm
Email Address []:ludovico.grabau@etu.hesge.ch
root@LAPTOP-31HU7CRF:/etc/apache2/ssl# openssl x509 -req -in fichier.csr -out fichier.crt -CA ca.crt -CAkey ca.key -CAcreateserial -CAserial ca.srl
Signature ok
subject=C = CH, ST = Geneva, L = Geneva, O = HEPIA, CN = Zboubelito, emailAddress = ludovico.grabau@etu.hesge.ch
Setting CA Private Key
Enter pass phrase for ca.key:
```

Troisième étape :

Signature du certificat serveur par l'autorité de certification

Signer la demande de certificat

→ `openssl x509 -req -in fichier.csr -out fichier.crt -CA ca.crt -CAkey ca.key -CAcreateserial -CAserial ca.srl`

6) A quoi sert l'option CAcreateserial ? Quand est-elle nécessaire ?

Ça nous permet de créer des certificats en série. Il est uniquement nécessaire lorsque c'est la première fois qu'on crée un certificat.

7) Que faut-il alors saisir ? Justifiez.

Par la suite on utilisera -CAserial à la place.

8) Quel est le certificat signé ? Quel est le résultat de sortie de la commande ?

Le certificat signé est fichier.crt.

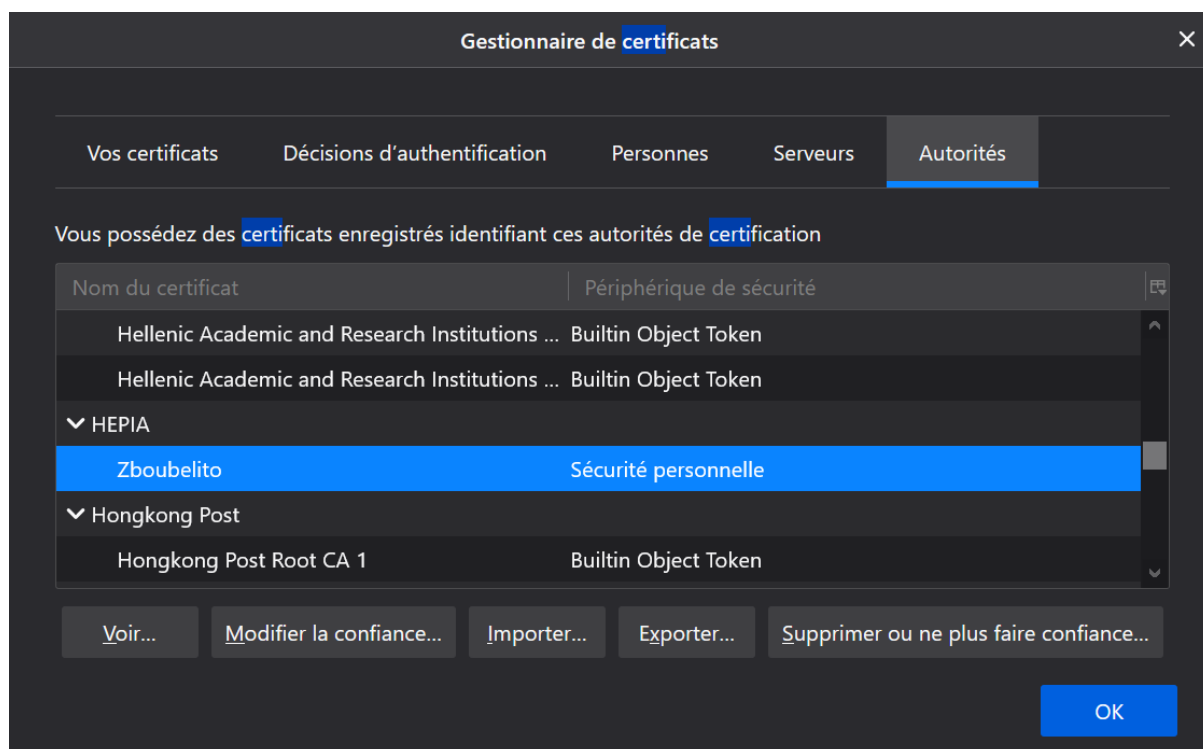
```
root@LAPTOP-31HJ7CRF:/etc/apache2/ssl# openssl x509 -req -in fichier.csr -out fichier.crt -CA ca.crt -CAkey ca.key -CAcreateserial -CAserial ca.srl
Signature ok
subject=C = CH, ST = Geneva, L = Geneva, O = HEPIA, CN = Zboubelito, emailAddress = ludovico.grabau@etu.hesge.ch
Setting CA Private Key
Enter pass phrase for ca.key:
```

Quatrième étape :

Installation du certificat d'autorité de certification

- Installer le certificat de l'autorité de certification dans le navigateur client qui validera le certificat reçu par le client lors de la requête web sécurisée
- Installer sur votre navigateur le certificat de l'autorité de certification, fichier ca.crt sous Mozilla Firefox

→ Voir https://www.dhimyotis.com/tuto/Navigateur/Firefox/importation_certificat_firefox.pdf



9) Quel est le protocole alors appelé/utilisé ?

C'est le protocole HyperText Transfert Protocol Secure (HTTPS).

Cinquième étape :

Installation et configuration d'Apache2

- Dans le fichier ports.conf le port 443 doit être en écoute

```
root@LAPTOP-31HU7CRF:/etc/apache2# cat ports.conf
# If you just change the port or add more ports here, you will likely also
# have to change the VirtualHost statement in
# /etc/apache2/sites-enabled/000-default.conf

Listen 80

<IfModule ssl_module>
    Listen 443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 443
</IfModule>

# vim: syntax=apache ts=4 sw=4 sts=4 sr noet
```

- Modifier le fichier de configuration par défaut de SSL (./sites-available/default-ssl.conf)
 Modifier : <VirtualHost _default_:443> en <VirtualHost *:443>
 Ajouter :
 ServerName « hostname » (inscrire le résultat de la commande “hostname”)
 Ajouter :
 SSLCertificateFile etc/apache2/ssl/fichier.crt
 SSLCertificateKeyFile etc/apache2/ssl/fichier.key
- Activer le site et redémarrer le processus d'apache2
 → a2ensite default-ssl
 → systemctl restart apache2

10) Que se passe-t-il ? A quoi sert cette commande ? (a2enmod ssl)

Cette commande permet d'initialiser un Secure Sockets Layer (SSL) à notre serveur web.

11) A quel service correspond ce numéro de port et comment voir que ce port est en écoute ?

Le port 443 correspond à la navigation sécurisée. Utile pour les connexions HTTP(S) utilisant une sécurité de type SSL.

On peut vérifier qu'il en écoute en allant à etc/apache2 et en faisant un « cat ports.conf » pour visualiser son contenu et voir qu'il est initialisé. On peut également utiliser la commande « netstat » avec « grep 443 » en filtre pour identifier s'il est actif.

12) Ajouter une ligne ServerName XXX (avec XXX le nom indiqué à l'étape de génération du CSR). A quoi sert cet ajout ?

C'est le nom qu'on donnera à notre serveur web sécurisé pour pouvoir y accéder par la suite.

13) Modifier <VirtualHost _default_:443> en <VirtualHost *:443>. A quoi cela sert ? Expliquer.

Cela permet de ne pas suivre l'exécution par défaut de apache2 qui permet d'accéder au site web <http://localhost>, nous permettant à la place d'utiliser le chemin sécurisé qu'on aura créé et mis en place à travers le TP dans le but d'accéder au serveur web d'apache2 avec `https://«ServerName»`. L'astérisque identifie donc qu'on utilise désormais les fichiers log customisés et d'authentications comme procédure de fonctionnement principale.

Sixième étape :

Phase de test :

A partir d'un navigateur sur un poste client tester le service : `https://<hostname>`
(Ecrire "hostname" dans le terminal d'Ubuntu pour visualiser le nom à écrire, celui-ci devrait le nom inscrit pour ServerName)

Conclusion

En guise de conclusion, nous avons réussi à créer un certificat serveur, un fichier de demande de signature de certificat, généré une clef privée pour ce dernier, créer un certificat d'autorité de certification, auto-signer le certificat, généré une clef privée pour mon autorité de certification, signer le certificat serveur par l'autorité de certification, importer le certificat d'autorité de certification sur Firefox, configurer apache2, le mettre en marche et accéder à son serveur web sécurisé.

Tout cela nous a donc permis de faire passer un serveur web utilisant HTTP à un sécurisé utilisant HTTPS grâce à l'utilisation d'un certificat serveur, d'un certificat d'autorité de certification, et de SSL.